

此页在源站不可用

第 1 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 2 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 3 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 4 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 5 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 6 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 7 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 8 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 9 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 10 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 11 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 12 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 13 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 14 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 15 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 16 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 17 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 18 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 19 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 20 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 21 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 22 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 23 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 24 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 25 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 26 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 27 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 28 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 29 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 30 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 31 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 32 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 33 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 34 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 35 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 36 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 37 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 38 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 39 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 40 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 41 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 42 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 43 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 44 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 45 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 46 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 47 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 48 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 49 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 50 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 51 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 52 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 53 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 54 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 55 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 56 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 57 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 58 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 59 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 60 页 / 共 129 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

- 除了encode()函数可以将字符串转换成bytes类型外，也可以自行定义字符串为bytes类型。
- 如下为示例代码：

```
#定义bytes类型变量  
B_Revenue = b"\xe8\x90\xa5\xe4\xb8"  
print(B_Revenue)
```



编码练习：

- ☁ 参照上面的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉二进制类型的相关知识点。



思考：

- ☁ 1.下面语句的输出结果是什么（）？可在Jupyter中输入后，验证输出结果。

```
x1=b"\\this is bytes example"  
print(x1)
```

- ☁ 2.将Python的常见数据类型整理成一张表，要求每种数据类型至少给出一个示例。

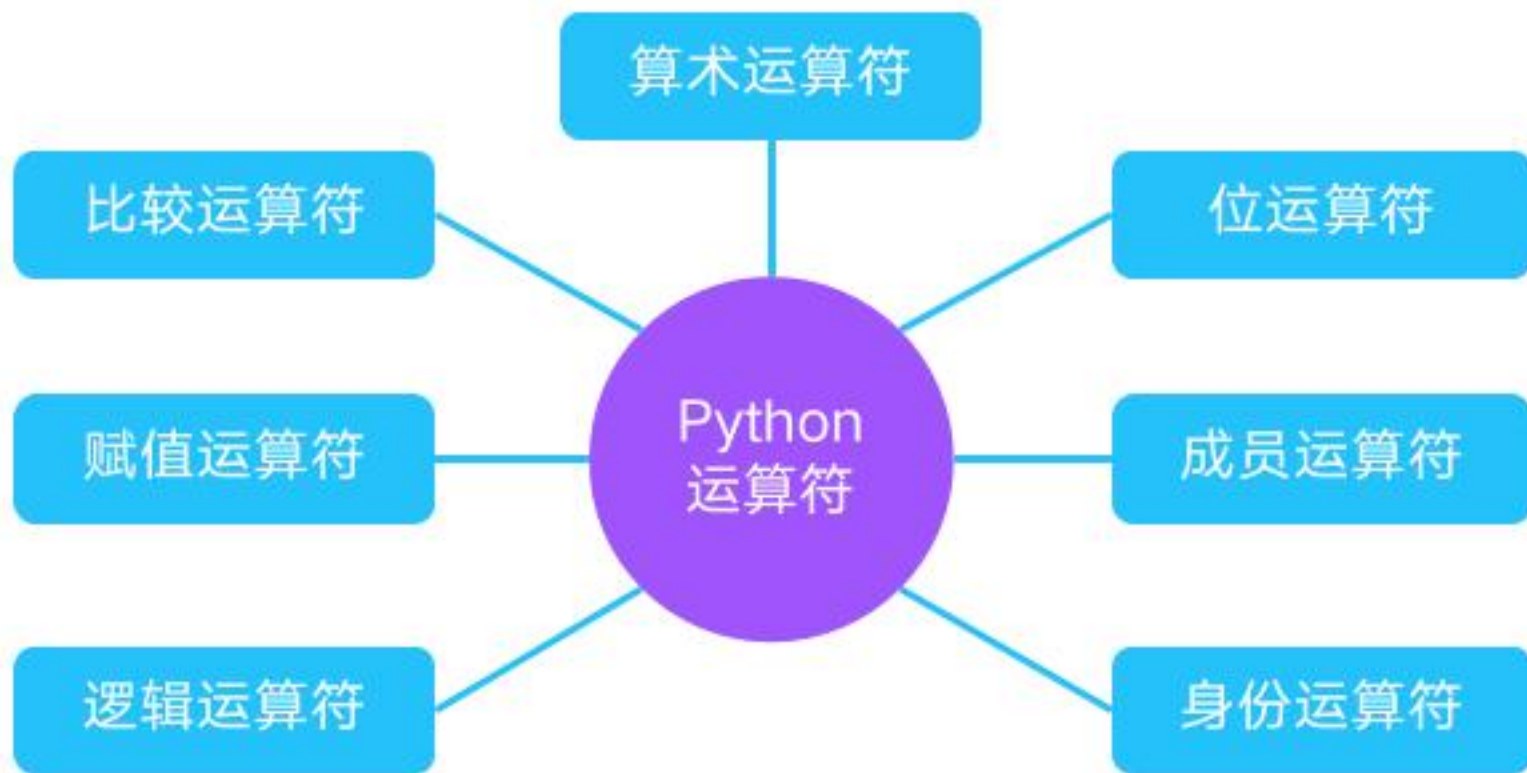


参考答案:

- ☁ 1. `b'\this is bytes example'`
- ☁ 2.略



☁ 在Python中常见的运算符：



- Python中常见的运算符有加、减、乘、除等
- 以下为算术运算符的示例：（假设Rc=8，Ro=24）

运算符	描述	示例
+	加 - 两个对象相加	Rc + Ro 输出结果 32
-	减 - 得到负数或是一个数减去另一个数	Rc - Ro 输出结果 -16
*	乘 - 两个数相乘 或是返回一个被重复若干次的字符串	Rc * Ro 输出结果 192
/	除 - Ro 除以 Rc	Ro / Rc 输出结果 3.0
%	取模 - 返回除法的余数	Ro % Rc 输出结果 0
**	幂 - 返回Rc的Ro次幂	Rc**Ro 为8的24次方
//	取整除 - 向下取接近商的整数	Ro//Rc为2



☁ 算术运算符对应的Python代码示例及运算结果：

#字符串与整数相乘的示例

`x="a"`

`y=3`

`print("x*y =", x*y)`

#取模%的示例

`x=34`

`y=5`

`print("x%y =", x%y)`

#取整除的示例

`x=12`

`y=5`

`print("正数 x//y =", x//y)`

`x=-12`

`y=5`

`print("负数 x//y =", x//y)`

`x*y = aaa`

`x%y = 4`

正数 `x//y = 2`

负数 `x//y = -3`



编码练习：

- ☁ 参照上面的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉算术运算符的相关知识点。



思考：

- ☁ 1.下面语句的输出结果是什么（）？可在Jupyter中输入后，验证输出结果。

```
x="abc"
y=4
print(x*y+x)
```

- ☁ 2.下面语句的输出结果是什么（）？要求先手工计算，再在Jupyter中输入，验证输出结果。

```
x=1
y=2
print(x**y+x//y+x/y+x%y)
```



参考答案:

- 1. 结果为: `abccabccabccabcc`
- 2. 结果为: `2.5`



- ☁ 比较运算符用于衡量运算符两边的关系，计算结果为布尔值（True/False）。
- ☁ 如果比较运算符返回1表示真，对应True，返回0表示假，对应False。
- ☁ 以下为比较运算符的示例：（假设Rc=8，Ro=24）

运算符	描述	示例
==	等于 - 比较对象是否相等	(Rc == Ro) 返回 False。
!=	不等于 - 比较两个对象是否不相等	(Rc != Ro) 返回 True。
>	大于 - 返回Rc是否大于Ro	(Rc > Ro) 返回 False。
<	小于 - 返回Rc是否小于Ro	(Rc < Ro) 返回 True。
>=	大于等于 - 返回Rc是否大于等于Ro	(Rc >= Ro) 返回 False。
<=	小于等于 - 返回Rc是否小于等于Ro	(Rc <= Ro) 返回 True。



- 在采购中，经常会判断实际采购价格与公司设定的标准采购价格之间的关系。
- 以下为比较运算符的示例：（假设Rc=8，Ro=24）

比较价格是否相等

#比较是否相等=的示例

```
P_采购价格=205
S_标准价格=200
if ( P_采购价格 == S_标准价格 ):
    print ("采购价格 等于 标准价格")
else:
    print ("采购价格 不等于 标准价格")
```

采购价格 不等于 标准价格

比较价格是否不等

#比较是否不等!=的示例

```
P_采购价格=200
S_标准价格=200
if ( P_采购价格 != S_标准价格 ):
    print ("采购价格 不等于 标准价格")
else:
    print ("采购价格 等于 标准价格")
```

采购价格 等于 标准价格



编码练习：

- ☞ 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉比较运算符的相关知识点。
- ☞ 2. 参照本节的语句样例，在Jupyter中编写代码，完成：
 - ☞ (1) >比较：根据需要，修改采购价格与标准价格的值，实现“>”比较运算。
 - ☞ (2) <比较：根据需要，修改采购价格与标准价格的值，实现“<”比较运算。
 - ☞ (3) >=比较：根据需要，修改采购价格与标准价格的值，实现“>=”比较运算。
 - ☞ (4) <=比较：根据需要，修改采购价格与标准价格的值，实现“<=”比较运算。



思考:

- ☁ 1.下面语句的输出结果是什么 () ? 要求先手工计算, 再在Jupyter中输入, 验证输出结果。

```
x=32  
y=35  
print((x==y)+6+(x!=y)+(x>y)+(x<=y))
```



参考答案:

☁ 1.结果为: 8



☁ 赋值运算符是将右边的值赋给左边的变量。

运算符	描述	示例
=	简单的赋值运算符	$c = a + b$ ，将 $a + b$ 的运算结果赋值为 c
+=	加法赋值运算符	$c += a$ 等效于 $c = c + a$
-=	减法赋值运算符	$c -= a$ 等效于 $c = c - a$
*=	乘法赋值运算符	$c *= a$ 等效于 $c = c * a$
/=	除法赋值运算符	$c /= a$ 等效于 $c = c / a$
%=	取模赋值运算符	$c \% = a$ 等效于 $c = c \% a$
**=	幂赋值运算符	$c ** = a$ 等效于 $c = c ** a$
//=	取整除赋值运算符	$c //= a$ 等效于 $c = c // a$
:=	海象运算符，可在表达式内部为变量赋值。用海象运算符可以重复调用语句两次，简化代码，后面有示例。	



- 案例：由于物料价格上涨，需要调整物料标准采购价格，将其上调1.2倍。
- 用赋值运算符*=实现，示例代码如下：

```
#赋值运算符 “*” 示例  
S_标准价格=200  
F_价格系数=1.2  
S_标准价格 *= F_价格系数  
print(S_标准价格)
```



- 案例：由于在标准成本计算时，多计算了人工成本，现在需要将多余的人工成本扣除。
- 用赋值运算符-=实现，示例代码如下：

```
#赋值运算符“-=”示例  
S_标准成本=309  
H_人工成本=12  
S_标准成本 -= H_人工成本  
print(S_标准成本)
```



- 案例：用于判断列表中的元素是否太多。
- 用传统代码和海象运算符:=实现，进行对比，示例代码如下。
- 海象运算符:=可用于将多条语句合并，简化语句，方便阅读和维护。

```
#传统的代码
my_list = [1,2,3,4,5]
count = len(my_list)
if count > 3:
    print("传统代码示例-错误! my_list中 元素太多")

# 当转换为海象运算符时...
if (count := len(my_list)) > 3:
    print("海象运算符示例-错误! my_list中 元素太多")
```



编码练习：

- ☞ 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉赋值运算符的相关知识点。
- ☞ 2. 参照本节的语句样例，在Jupyter中编写代码，完成：
 - ☞ (1) =运算：原来的采购价格=200，新的采购价格=206，需更新采购价格，要求编码实现“=”赋值运算。
 - ☞ (2) +=运算：原来的采购价格=200，现增加检验费20，需更新采购价格，要求编码实现“+=”赋值运算。
 - ☞ (3) /=运算：原来的采购价格=200，谈判后为原来的1/1.05，需更新采购价格，要求编码实现“/=”赋值运算。
 - ☞ (4) %=运算：a=20，b=9，用a%=b更新a，编码实现“%=”赋值运算。
 - ☞ (5) **=运算，a=20，b=2，用a**=b更新a，编码实现“**=”赋值运算。



参考答案:

2 (1) 代码:

```
#赋值运算符=示例  
采购价格=200  
新采购价格=206  
采购价格=新采购价格  
print(采购价格)
```

(2) 代码:

```
#赋值运算符+=示例  
采购价格=200  
检验费=20  
采购价格+=检验费  
print(采购价格)
```

(3) 代码:

```
#赋值运算符/=示例  
采购价格=200  
价格系数=1.05  
采购价格/=价格系数  
print(采购价格)
```

(4) 代码:

```
#赋值运算符%=示例  
a=20  
b=9  
a%=b  
print(a)
```

(5) 代码:

```
#赋值运算符**=示例  
a=20  
b=2  
a**=b  
print(a)
```



思考:

1. 下面语句的输出结果是什么 () ? 要求先手工计算, 再在Jupyter中输入, 验证输出结果。

```
a=4  
b=2  
a+=b  
a/=b  
a**=b  
a%=b  
print(a)
```

2. 下面语句的输出结果是什么?

```
a=12  
b=8  
print(a+=b)
```



参考答案：

- ☁ 1. 结果为：1.0。
- ☁ 2. 系统会报错，提示语法错误。



☁ 逻辑运算符是两个变量进行逻辑运算。

运算符	表达式	描述
and	x and y	布尔"与" - 如果 x 为 False, x and y 返回 x 的值, 否则返回 y 的计算值。
or	x or y	布尔"或" - 如果 x 是 True, x or y 返回 x 的值, 否则返回 y 的计算值。
not	not x	布尔"非" - 如果 x 为 True, 返回 False 。如果 x 为 False, 返回 True。



在下面的and运算代码示例中，有两个关键点先说明：

- ☁ bool(a)为布尔函数，用于判断a的布尔值为True还是False
- ☁ 如果变量的值为0，其布尔值就为False；如果不为0，其布尔值就为True
- ☁ 下面第一个示例中：a and b 为：True and True，返回b
- ☁ 第二个示例中，a and b为：False and True，返回a

```
# 逻辑运算 and示例
# a不为False, 返回b
a = 1
b = 2
print(bool(a), bool(b), (a and b))

# a为False, 返回a
a = 0
b = 9
print(bool(a), bool(b), (a and b))
```

```
True True 2
False True 0
```



下面为or运算代码示例，有两个关键点先说明：

- 下面第一个示例中：a or b 为：True or True，返回a
- 第二个示例中，a or b为：False or True，返回b

```
# 逻辑运算 or示例
# a是True, 返回a
a = 1
b = 2
print(bool(a), bool(b), (a or b))

# a不为True, 返回b
a = 0
b = 9
print(bool(a), bool(b), (a or b))
```

```
True True 1
False True 9
```



下面为not运算代码示例，有两个关键点先说明：

- ☞ a=3, bool(a)=True, 所以(not a)返回False。
- ☞ a=0, bool(a)=False, 所以(not a)返回True。

```
# 逻辑运算 not示例  
# a是True, 返回False  
a = 3  
print(bool(a), (not a))  
  
# a是False, 返回True  
a = 0  
print(bool(a), (not a))
```

```
True False  
False True
```



编码练习：

- ☁ 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉逻辑运算符的相关知识点。
- ☁ 2. 参照本节的语句样例，在Jupyter中编写代码，完成：
 - ☁ (1) and运算：a=6，b=0，要求编码实现 a and b的运算。
 - ☁ (2) and运算：a=0，b=0，要求编码实现 a and b的运算。
 - ☁ (3) or运算：a=6，b=0，要求编码实现 a or b的运算。
 - ☁ (4) or运算：a=0，b=0，要求编码实现 a or b的运算。



参考答案:

2. (1) 代码:

```
# a不为False, 返回b  
a = 6  
b = 0  
print(bool(a),bool(b),(a and b))
```

(2) 代码:

```
# a为False, 返回a  
a = 0  
b = 0  
print(bool(a),bool(b),(a and b))
```

(3) 代码:

```
# a为True, 返回a  
a = 6  
b = 0  
print(bool(a),bool(b),(a or b))
```

(4) 代码:

```
# a为False, 返回b  
a = 0  
b = 0  
print(bool(a),bool(b),(a or b))
```



思考:

- ☁ 1.下面语句的输出结果是什么 () ? 要求先手工计算, 再在Jupyter中输入, 验证输出结果。

```
a=3  
b=9  
print((not (a and b)) or a)
```

- ☁ 2.下面语句的输出结果是什么?

```
a="dd"  
b=4  
print(a and b)
```



参考答案：

- ☁ 1. 结果为：3。
- ☁ 2. 结果为：4，由本题可以得出：字符串的布尔值为True



- 按位运算符是把数字看作二进制来进行计算。
- 假设a=1(其二进制为01), b=2(其二进制为10), 查看下表:

运算符	描述	示例
&	按位与运算符: 参与运算的两个值,如果两个相应位都为1,则该位的结果为1,否则为0	(a & b) 输出结果 0, 二进制运算: 01 & 10 = 00
	按位或运算符: 只要对应的二个二进位有一个为1时, 结果位就为1。	(a b) 输出结果 3, 二进制运算: 01 10 = 11
^	按位异或运算符: 当两对应的二进位相异时, 结果为1	(a ^ b) 输出结果 3, 二进制运算: 01 ^ 10 = 11
~	按位取反运算符: 对数据的每个二进制位取反,即把1变为0,把0变为1。~x 类似于 -x-1	(~a) 输出结果 -2, 二进制运算: ~ +01 = -10, 为有符号二进制数的补码形式。
<<	左移动运算符: 运算数的各二进位全部左移若干位, 由"<<"右边的数指定移动的位数, 高位丢弃, 低位补0。	a << 1 输出结果 2, 二进制运算: 01 << 1 = 10
>>	右移动运算符: 把">>"左边的运算数的各二进位全部右移若干位, ">>"右边的数指定移动的位数	a >> 1 输出结果 0, 二进制运算: 01 >> 1 = 00

- 右边为位运算的示例代码：
- $a \& b$ ，即：01 & 10，对应为首位的0和1相与，两者有一个为0，故其值为0；第二位的1和0相与，两者有一个为0，故其值也为0，故01 & 10 = 00，对应十进制的0。
- $a | b$ ，即：01 | 10，对应为首位的0和1相或，两者有一个为1，故其值为1；第二位的1和0相或，两者有1个为1，故其值也为1，故01 | 10 = 11，对应十进制的3。
- $a \wedge b$ ，即：01 ^ 10，对应为首位的0和1相异或，因两者相异，故其值为1；第二位的1和0相异或，因两者相异，故其值也为1，故01 ^ 10 = 11，对应十进制的3。
- $\sim a$ ，即： ~ 01 ，首先符号取反，为负数；首位的0取反，变为1；第二位的1取反，其值为0，故 $\sim 01 = -10$ ，对应十进制的-2。
- $a \ll 1$ ，即：01 << 1，首位的0左移一位，超出范围，去掉；第二位的1也左移一位，后面空一位，用0来填充，故01 << 1 = 10，对应十进制的2。
- $a \gg 1$ ，即：01 >> 1，第二位的1右移一位，超出范围，去掉；首位的0也右移一位，后面空一位，用0来填充，故01 >> 1 = 00，对应十进制的0。

#位运算示例

```
a=1
b=2
print("a&b=", a&b, "a|b=", a|b, "a^b=", a^b,)
print("~a=", ~a, "a<<1=", a<<1, "a>>1=", a>>1)
```

```
a&b= 0 a|b= 3 a^b= 3
~a= -2 a<<1= 2 a>>1= 0
```



编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉位运算符的相关知识点。



思考:

1. 下面语句的输出结果是什么 () ? 要求先手工计算, 再在Jupyter中输入, 验证输出结果。

```
a=3  
b=1  
print(((a&b)|b)<<1)
```

2. 下面语句的输出结果是什么? 要求先手工计算, 再在Jupyter中输入, 验证输出结果。

```
a=2  
b=3  
print(((a>>1)|b)^a)
```



参考答案：

☁ 1. 结果为：2。

☁ 2. 结果为：1。



- 成员运算符主要是针对多元素类的数据类型，用于判断某个元素是否在序列中。
- 该类的数据类型包括：字符串、列表、元组、字典、集合等。
- 以下为说明，假设：a=1，b=[1,2,3]。

运算符	描述	示例
in	如果在指定的序列中找到值，返回 True，否则返回 False。	a 在 b 序列中,返回 True。
not in	如果在指定的序列中没有找到值，返回 True，否则返回 False。	a 在 b 序列中,返回 False。

成员运算符的代码示例：

```
# in示例  
a="营业收入"  
b=["营业收入","利息收入","已赚保费","手续费及佣金收入"]  
print("a in b=",a in b)  
  
# not in 示例  
c="营业成本"  
b=["营业收入","利息收入","已赚保费","手续费及佣金收入"]  
print("c not in b=",c not in b)
```

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉位成员运算符的相关知识点。



思考:

☁ 1.下面语句的输出结果是什么 () ?

```
a=2  
b=[8,3,4,5,(3,2,6)]  
print("a in b=",a in b)
```



参考答案:

☞ 1. false



- 身份运算符主要用于判断两个元素的身份标识是否相同，即id是否相同。
- 在python中，一个变量有三个要素：类型、身份标识、值。以 $x=3$, $y=3$ 为例：
- 类型：指变量的类型。 $x=3$ 中， x 类型为int。
- 身份标识：指变量的地址。如果 $x=3$, $y=x$ ，这时就可以说 x 和 y 的地址相同，两者的身份标识相同，即 x is y 。
- 值：指变量的值，两个变量的值是否相同，一般用 $==$ 来判断。如： $x=3$, $y=3$ ，则 $x==y$ 。



☞ 以下为身份运算符的说明：

运算符	描述	示例说明
is	is 是判断两个标识符是不是引用自一个对象	x is y, 类似 <code>id(x) == id(y)</code> ，如果引用的是同一个对象则返回 True，否则返回 False
is not	is not 是判断两个标识符是不是引用自不同对象	x is not y，类似 <code>id(x) != id(y)</code> 。如果引用的不是同一个对象，则返回结果 True，否则返回 False。



对右边身份运算符的代码说明：

- 针对所有的数据类型，两个变量的值相同时，`x==y`都为True，说明判断两个变量的值是否相等，用`==`比较运算符是合适的。
- 当变量的类型为int、字符串时，两个变量的值相等，其 `x is y`的结果也是True。这是因为整型和字符串都是简单对象，Python 认为这些简单对象会经常被调用，故很高效地对它们进行缓存。即当新增一个变量y时，如果y和x的值相同，不会单独为y创建新地址，而是与x共用相同的地址，因此`x is y`的结果为True。

```
# is 示例: int类型时
x=20
y=20
print("x==y=", x==y, "x is y=", x is y)

# is 示例: 字符串类型时
x="123"
y="123"
print("x==y=", x==y, "x is y=", x is y)

# is 示例: 列表类型时
x=[1, 2, 3]
y=[1, 2, 3]
print("x==y=", x==y, "x is y=", x is y)

# is 示例: 元组类型时
x=(1, 2, 3)
y=(1, 2, 3)
z=x
print("x==y=", x==y, "x is y=", x is y, "x is z=", x is z)

x==y True x is y True
x==y True x is y True
x==y True x is y False
x==y True x is y False x is z True
```

对右边身份运算符的代码说明：

- 当变量的类型为列表、元组时，两个变量的值相等，但其 `x is y` 的结果是 `False`。列表、元组为较复杂的对象，若新增变量 `y`，在 `y` 值与 `x` 值相同时，不会像 `int` 类型那样，让 `y` 与 `x` 共用地址，而是会单独为 `y` 新增地址，因此 `x is y` 的结果就为 `False`。
- 在最后一语句中，`z=x`，表示 `z` 与 `x` 是完全相同的，将 `x` 的值赋给 `z` 的同时，`z` 与 `x` 也将共用地址，因此 `x is z` 的结果为 `True`。

```
# is 示例: int类型时
x=20
y=20
print("x==y=", x==y, "x is y=", x is y)

# is 示例: 字符串类型时
x="123"
y="123"
print("x==y=", x==y, "x is y=", x is y)

# is 示例: 列表类型时
x=[1, 2, 3]
y=[1, 2, 3]
print("x==y=", x==y, "x is y=", x is y)

# is 示例: 元组类型时
x=(1, 2, 3)
y=(1, 2, 3)
z=x
print("x==y=", x==y, "x is y=", x is y, "x is z=", x is z)
```

```
x==y= True x is y= True
x==y= True x is y= True
x==y= True x is y= False
x==y= True x is y= False x is z= True
```

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉身份运算符的相关知识点。



思考:

☁ 1.下面语句的输出结果是什么 () ?

```
# is 示例: 集合类型时  
x={1,2,3,4,5}  
y={1,2,3,4,5}  
print("x==y=",x==y,"x is y=",x is y)
```

☁ 2.下面语句的输出结果是什么 () ?

```
# is not 示例: dict类型时  
x={"a":1,"b":2,"c":3}  
y={"a":1,"b":2,"c":3}  
print("x==y=",x==y,"x is not y=",x is not y)
```



参考答案:

- 1. $x==y = \text{True}$ $x \text{ is } y = \text{False}$
- 2. $x==y = \text{True}$ $x \text{ is not } y = \text{True}$



☞ 数字格式化是将数字以期望的格式输出。

☞ 以下为格式说明：

数字	格式	输出	描述
3.1415926	{:.2f}	3.14	保留小数点后两位
3.1415926	{:+.2f}	+3.14	带符号保留小数点后两位
-1	{:-.2f}	-1.00	带符号保留小数点后两位
2.71828	{:.0f}	3	不带小数
5	{:0>2d}	05	数字补零 (填充左边, 宽度为2)
5	{:x<4d}	5xxx	数字补x (填充右边, 宽度为4)
10	{:x<4d}	10xx	数字补x (填充右边, 宽度为4)
1000000	{:,}	1,000,000	以逗号分隔的数字格式
0.25	{:.2%}	25.00%	百分比格式
1000000000 0	{:.2e}	1.00e+09	指数记法
13	{:>10d}	13	右对齐 (默认, 宽度为10)
13	{:<10d}	13	左对齐 (宽度为10)
13	{:^10d}	13	中间对齐 (宽度为10)

右边代码示例说明：

- print("{:.2f}".format(x))中，{:.2f}是指定输出的格式，为保留2位小数。format(x)函数对变量按照格式要求进行处理，print()输出最终的结果。
- print("{:,}".format(x))中，{:,}是对长数字按照千分位的方式，用逗号进行分隔，便于阅读。
- print("{:.2%}".format(x))中，{:.2%}是对数字按照百分比的格式输出，保留2位小数。
- print("{:.2e}".format(x))中，{:.2e}是对数字按照指数记法格式输出，保留2为小数。

```
# 将数字保留2位小数输出
x=3.1415926
print("{:.2f}".format(x))

# 将长数字用逗号分隔后输出
x=329685069385
print("{:,}".format(x))

# 将数字保留2位小数，按百分比输出
x=0.329685069385
print("{:.2%}".format(x))

# 将数字保留2位小数，按指数格式输出
x=329685069385
print("{:.2e}".format(x))
```

```
3.14
329,685,069,385
32.97%
3.30e+11
```

- format()函数也可以传入参数，来实现灵活的输出。
- 以下为示例代码的说明：
 - Income是dict数据类型，其中包含有两对值，分别用name、revenue来标识。
 - print("项目: {name} 值: {revenue}".format(**Income))中，"项目: {name} 值: {revenue}"是字符串，但其中包含有变量，变量的值用Income中的元素来替换。
 - format(**Income)中有两个*，这是针对dict类型变量的调用书写规范。

```
#format()设置参数的示例
Income = {"name": "营业收入", "revenue": 58934068.0}
print("项目: {name} 值: {revenue}".format(**Income))
```

项目: 营业收入 值: 58934068.0

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉format()的相关知识点。



思考:

- ☞ 1.下面语句的输出结果是什么 () ?

```
print('{:02}:{:02}:{:02}'.format(22,2,56))
```

- ☞ 2.下面语句的输出结果是什么 () ?

```
list=['营业收入','利润表']  
print('{names[0]} 是 {names[1]} 中的一个项目'.format(names=list))
```



参考答案：

- ☁ 1. 结果为：22:02:56。解析：{:02}:{:02}:{:02}是将后面的数据按照指定长度的格式输出。在format(22,2,56)中有三个参数，要求都按照2位输出，22和56都是2位，故保持不变，第二个数据2在输出时就转换为：02，故整体输出为：22:02:56。
- ☁ 2. 结果为：营业收入 是 利润表 中的一个项目。解析：list为列表类型，此处为列表类型的调用方法。names=list是将list中的元素赋给变量组names，分别对应names[0]，names[1]。故names[0]= '营业收入'，names[1]= '利润表'，将两个变量替换后，输出结果为：营业收入 是 利润表 中的一个项目。



- print()函数的语法:
- `print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`
- 参数*objects是指输出的对象，一次输出可以是多个对象，对象之间用逗号“,”分隔
- 参数sep用来设置分隔符
- 参数end用来设定以什么结尾。默认值是换行符 `\n`
- 参数file是指要写入的文件对象
- 参数flush用来决定输出是否被缓存。



- print()函数的语法:
- `print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`
- 参数*objects是指输出的对象，一次输出可以是多个对象，对象之间用逗号“,”分隔
- 参数sep用来设置分隔符
- 参数end用来设定以什么结尾。默认值是换行符 `\n`
- 参数file是指要写入的文件对象
- 参数flush用来决定输出是否被缓存。



print()函数的示例代码说明:

- print("www","finance","com","cn",sep=".")中, 是用分隔符 “.” 将多个元素分开, 因此输出的就是网址的格式。
- print("收入项目 营业收入 利息收入 |",end="")中, end设置为空, 因此不换行, 下一个print()语句仍旧在同一行输出。
- print("成本项目 营业成本 手续费及佣金支出",end="\n")中, end设置为\n, 表示要换行, 下一个print()语句就在下一行输出。

```
#使用间隔符号.  
print("www","finance","com","cn",sep=".")  
  
#end设置为空  
print("收入项目 营业收入 利息收入 |",end="")  
#end设置为\n  
print("成本项目 营业成本 手续费及佣金支出",end="\n")  
print("收入项目 营业收入 利息收入 |",end="")
```

www.finance.com.cn

收入项目 营业收入 利息收入 |成本项目 营业成本 手续费及佣金支出
收入项目 营业收入 利息收入 |

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉print()的相关知识点。



思考:

- ☁ 1.下面语句的输出结果是什么 () ?

```
print("收入项目","营业收入","利息收入","已赚保费",sep="|")
```



参考答案：

- ☁ 1. 结果为：收入项目|营业收入|利息收入|已赚保费。



1 案例背景

2 知识准备

3 任务要求

4 任务解析

华盛科技公司有三家竞争对手，分别是A公司、B公司、C公司。若已知三家企业上年利润分别是30万、20万、25万，华盛科技的利润是三家竞争对手公司上年利润总和的一半，请输出这四家公司的上年利润，以及各家公司上年利润占四个公司的利润总和的比率，要求自行设计输出格式，方便使用者阅读。



1 案例背景

2 知识准备

3 任务要求

4 任务解析

思考：

☁ 针对上面的案例背景，请列出主要的解题思路，并分享讨论。



参考答案：

- ⇒ A公司、B公司、C公司、华盛科技的利润分别用四个变量a、b、c、d来表示，列出表达式为：
- ⇒ $a = 30$
- ⇒ $b = 20$
- ⇒ $c = 25$
- ⇒ $d = (a+b+c)/2$
- ⇒ 各家公司所占利润的比率，用变量：e、f、g、h来表示，列出表达式为：
- ⇒ $e = (a/(a+b+c+d)) * 100\%$
- ⇒ $f = (b/(a+b+c+d)) * 100\%$
- ⇒ $g = (c/(a+b+c+d)) * 100\%$
- ⇒ $h = (d/(a+b+c+d)) * 100\%$
- ⇒ 按格式输出时，可以用print()函数结合format()函数来实现。



参考代码（未调整输出格式）：

```
#四家公司的利润
a = 30
b = 20
c = 25
d = (a+b+c)/2

#四家公司的利润占比：
e =(a/(a+b+c+d))
f =(b/(a+b+c+d))
g =(c/(a+b+c+d))
h =(d/(a+b+c+d))

print("华盛科技的利润为：", d)
print("四家公司的利润占比")
print("A公司：", e, " | B公司：", f)
print("C公司：", g, " | 华盛科技：", h)
```

华盛科技的利润为： 37.5

四家公司的利润占比

A公司： 0.26666666666666666 | B公司： 0.17777777777777778

C公司： 0.22222222222222222 | 华盛科技： 0.33333333333333333



参考代码（调整输出格式）：

```
#四家公司的利润
a = 30
b = 20
c = 25
d = (a+b+c)/2

#四家公司的利润占比:
e = (a/(a+b+c+d))
f = (b/(a+b+c+d))
g = (c/(a+b+c+d))
h = (d/(a+b+c+d))

print("_____")
print("华盛科技的利润为：", d, "万元")
print("_____")
print("四家公司的利润占比")
print("_____")
print("A公司：", "{:.2%}".format(e), " |   B公司：", "{:.2%}".format(f))
print("C公司：", "{:.2%}".format(g), " |   华盛科技：", "{:.2%}".format(h))
print("_____")
```

华盛科技的利润为： 37.5 万元

四家公司的利润占比

A公司：	26.67%		B公司：	17.78%
C公司：	22.22%		华盛科技：	33.33%

1.华盛科技公司共有四大片区，西部片区、南部片区、东部片区、北部片区，每个片区年度销售额分别为60万、130万、150万、100万，请输出华盛科技有限公司年度销售额。

2.华盛科技有限公司去年的销售额为400万，今年计划完成的销售额为500万。公司分为四大片区：西部片区、南部片区、东部片区、北部片区，各片区今年计划完成销售额分别为80万、100万、120万、100万，今年四个片区实际完成的销售额分别为60万、130万、150万、100万。请输出华盛科技有限公司今年的销售计划完成率、四个片区的销售计划完成率及今年的销售增长率。要求自行设计输出格式，方便使用者阅读。

3.现有华盛科技公司和宏达科技公司两家公司，已知华盛科技公司的流动负债为50万元，总负债200万元，流动资产为20万元，总资产500万元，存货为10万元；宏达科技公司的流动负债为30万元，总负债50万元，流动资产为50万元，总资产100万元，存货为40万元；请输出华盛科技公司和宏达科技公司的资产负债率和速动比率，并比较两家公司的资产负债率。要求自行设计输出格式，方便使用者阅读。



感謝
ขอบคุณ
terima kasih
ありがとう
Thanks
谢谢